

TenantFlow Architecture Report

v0.7.0 Engineering Phase: Dynamic Multi-Tenant Schema Routing Blueprint

1. Executive Summary

This blueprint details the structural transformation of the **TenantFlow** infrastructure from a shared-schema column discriminator model to an enterprise-grade **Dynamic Multi-Tenant Schema Routing** topology. By partitioning tenant data into discrete physical database schemas, the system achieves maximum data isolation compliance, eliminates cross-tenant access risks, and ensures future horizontal scalability.

2. Current vs. Target Architectural Topology

The architectural transition shifts validation logic away from the runtime application querying filters directly down to the database management level:

Metric / Feature	v0.6.0 (Shared Schema)	v0.7.0 (Discrete Schema Routing)
Isolation Layer	Logical (SQL row-level grouping via WHERE)	Physical (PostgreSQL discrete Schema separation)
Query Mechanics	Manual/Automated method binding filter predicates	Dynamic runtime connection routing parameters
Data Leak Risk	Low (Guarded via AOP interceptors)	Zero (Cryptographically decoupled boundaries)
Backup Scalability	Complex (Row parsing required)	Native (Simple schema dump commands)

3. Core Implementation Strategy

To achieve dynamic routing without dropping active server connections, TenantFlow will leverage Hibernate 6's native multi-tenancy extension hooks:

Step 1: Tenant Resolver Integration

Implement `CurrentTenantIdentifierResolver` to bridge the gap between Spring Security context and the Hibernate transaction context. This component resolves the active tenant token from the thread-scoped container populated during the initial request lifecycle.

Step 2: Dynamic Connection Provider

Implement `MultiTenantConnectionProvider`. Instead of spawning isolated pool instances per tenant, a unified master connection pool (HikariCP) will fetch active connections, subsequently altering the isolation context using native database routing handles:

```
ALTER ROLE CURRENT_USER SET search_path TO tenant_schema;
```

4. Next Implementation Milestones

- Configure the Spring Liquibase orchestration engine to handle dynamic multi-schema DDL migrations during application boots.

- Refactor database initialization files to support decoupled schemas alongside shared public tables.